Understand Anything 从入门到精通

> **版本与核实信息**
>
> - 项目：Understand Anything，开源仓库为 `Egonex-AI/Understand-Anything`（GitHub），MIT 许可证，版权归属 Yuxiang Lin 与 Infinite Universe, Inc.
> - 官方主页：`https://understand-anything.com`；在线交互演示：`https://understand-anything.com/demo/`
> - 核实日期：2026 年 10 月 23 日。本教程依据该日期仓库 main 分支的 README、`install.sh`、`install.ps1`、`package.json` 与 `understand-anything-plugin/skills` 目录清单逐一核实撰写。
> - 项目未在 `package.json` 中声明版本号，采用 main 分支持续更新模式。教程涉及的平台兼容性、命令名与安装产物均以官方仓库为准；版本相关的行为差异在正文出现处单独标注。
> - 核实条件说明：撰写期间通用搜索引擎不可用，全部事实内容来自官方仓库页面与原始文件的直接获取；无法核实的细节一律省略或标注「以官方文档为准」。

前言

本教程面向需要快速理解陌生代码库的开发者、技术负责人与新团队入职人员，覆盖从安装、首次分析，到团队共享、业务逻辑提取，再到原理掌握与项目贡献的完整学习路径。阅读本教程前，读者应具备基本的命令行操作经验与 Git 使用经验；是否使用过 AI 编码助手不构成硬性前提，教程将在相应章节说明。

写作约定如下。第一，两类调用形式并存于本教程：Claude Code 等命令式平台以斜杠书写，例如 `/understand`，Codex 平台以 `$` 为前缀；OpenCode 中技能经 agent 会话加载与触发，以 `@` 提及或自然语言请求实现（详见 2.5 节），各章示例中在 OpenCode 场景下的等价表达以 2.5.3 节对照表为准。第二，命令示例区分 macOS/Linux（bash）与 Windows（PowerShell）两种环境，路径分别以 `$HOME` 与 `%USERPROFILE%` 表示。第三，官方文档未固定展示控制台输出的完整样例，本教程不虚构输出内容，凡涉及具体输出处均注明「以实际运行为准」。第四，中英文与数字之间保留一个空格。

第一篇 入门篇

第 1 章 认识 Understand Anything

刚加入一个团队，面对的是一个二三十万行的仓库；文档滞后、命名混乱、依赖关系只存在于资深同事的头脑中。了解「这个系统到底如何运转」的传统方式是逐文件通读源码，而 Understand Anything 提供了另一种路径：先把整个代码库变成一张可以浏览、搜索、提问的知识图谱，再按需深入。

1.1 定位：面向 AI 编码助手的知识图谱技能

Understand Anything 是一个安装在 AI 编码助手（如 Claude Code、OpenCode、Codex、Gemini CLI 等）中的技能插件。安装后，助手获得一组以 `understand` 为名的技能指令；执行分析指令时，助手按内置流程调用多个专项智能体（agent，指承担单一职责的被调度执行单元），对项目完成结构解析与语义总结，生成知识图谱数据，并可通过浏览器以交互式面板（本教程称 Dashboard）浏览。

在此产品语境下，**知识图谱（knowledge graph）**指一类结构化数据：以文件、函数、类、依赖等代码元素为节点，以引用、调用、继承等关系为边，同时为每个节点附带自然语言摘要。项目自己的定位表述值得注意：图谱的目的不是展现代码库多么复杂，而是安静地教会读者各部分如何拼合（原文：Graphs that teach > graphs that impress）。

1.2 三层工作结构

Understand Anything 的运行由三层构成，后续章节均围绕这三层展开。

层	说明
分析流水线	多智能体流水线：静态解析引擎抽取结构事实，LLM 生成语义摘要（详见第 9 章）
图谱数据	分析结果保存为项目内的 JSON 文件，默认位于 `.ua/` 目录，主编谱文件为 `.ua/knowledge-graph.json`
交互式 Dashboard	浏览器中的可视化面板，提供结构图、搜索、导览、角色视角等能力（详见第 4 章）

这种分层带来一个重要性质：图谱只是 JSON 文件，可以提交进 Git 与团队共享，队友无需重复分析即可浏览（详见第 7 章）。

1.3 核心概念

以下术语在后文反复出现，此处给出统一定义。

术语	定义
节点与边	图谱的基本组成：节点为文件、函数、类等元素；边为节点间的引用、调用、继承等关系
导览（Guided Tours）	按依赖顺序自动生成的架构讲解路径，限定按合理顺序学习代码库
架构分层	自动归纳的层次分组，包含 API、Service、Data、UI、Utility 五类，在 Dashboard 中以颜色区分
业务域	系统承载的业务领域划分，含业务流程（flow）与流程步骤（step），由 `/understand-domain` 生成
语言概念	出现在代码中的编程模式（如泛型、闭包、装饰器）的顺文解释，共收录 12 种模式
角色自适应界面	Dashboard 依据使用者角色（初级开发、项目经理、重度用户）调节信息详略程度
增量更新	重复分析时仅重新处理有变更的文件，控制时间与 token 开销

1.4 适用与不适用场景

适用场景包括：新成员入职引导、接手陌生项目、评审前了解影响面、将代码结构文档化为团队资产，以及让 LLM 助手获得对项目的全局认知。不适用或需调整预期的场景包括：纯文档类知识库应使用 `/understand-knowledge`（见第 8 章）；对 token 成本高度敏感的团队应评估本地模型方案（见第 2 章与第 10 章）。

本章要点：Understand Anything 不是独立软件，而是安装在 AI 编码助手内的技能；其输出是可共享的 JSON 图谱与浏览器面板；理解节点、边、导览、分层四个概念即可开始动手。

第 2 章 安装与平台接入

Understand Anything 支持十余个 AI 编码平台，但接入方式分三路：Claude Code 走原生插件市场，多数 CLI 平台走一键安装脚本，Cursor 与 VS Code 走仓库自动发现。本章先给出平台矩阵与通用方法，再重点展开 OpenCode 的完整接入步骤。

2.1 环境要求与平台兼容矩阵

所有平台共享两个前提：一是所使用的 AI 编码助手本身已安装并可联网调用模型；二是本机已安装 Git（安装脚本通过 Git 克隆仓库）。后文第 7 章的免分析浏览通道另要求 Node.js 18 或更高版本。

以下矩阵为核实日期当日官方 README 所列内容。

平台	接入方式	备注
Claude Code	插件市场（原生）	推荐，命令与智能体支持最完整
OpenCode	一键安装脚本	本章 2.4 节详解
Codex	一键安装脚本	调用前缀为 `$` 而非 `/`
Cursor	仓库克隆后自动发现	备用手动添加
VS Code + GitHub Copilot	仓库克隆后自动发现	要求 Copilot v1.108 或更高
Copilot CLI	插件安装命令	见 2.6 节
Gemini CLI、Pi Agent、OpenClaw、Antigravity、Vibe CLI、Hermes、Cline、KIMI CLI、Trae、Nanobot、Kiro	一键安装脚本	平台 ID 见 2.6 节

安装脚本的平台 ID 全集为：`gemini`、`codex`、`opencode`、`pi`、`openclaw`、`antigravity`、`vibe`、`vscode`、`hermes`、`cline`、`kimi`、`trae`、`nanobot`、`kiro`。

2.2 Claude Code 原生安装

在 Claude Code 中依次执行以下两条命令。第一条将官方仓库注册为插件市场来源，第二条安装名为 `understand-anything` 的插件。

```
/plugin marketplace add Egonex-AI/Understand-Anything
/plugin install understand-anything
```

预期行为：命令完成后插件进入可用状态；以实际运行为准，官方未固定输出样例。计划使用本地模型的团队请先阅读 3.1 节再决定模型提供方。

2.3 多平台一键安装

一键安装脚本做三件事：将仓库克隆到 `~/.understand-anything/repo`（Windows 为 `%USERPROFILE%\.understand-anything\repo`）；按所选平台把技能链接进对应目录；创建一个指向插件根目录的通用链接 `~/.understand-anything-plugin`。脚本同时支持交互式选择平台与传参指定平台。

macOS 与 Linux：

```
curl -fsSL https://raw.githubusercontent.com/Egonex-AI/Understand-Anything/main/install.sh | bash
# 传参指定平台，跳过交互提示（以 OpenCode 为例）
curl -fsSL https://raw.githubusercontent.com/Egonex-AI/Understand-Anything/main/install.sh | bash -s opencode
```

Windows（PowerShell，交互式选择平台）：

```
iwr -useb https://raw.githubusercontent.com/Egonex-AI/Understand-Anything/main/install.ps1 | iex
```

安装完成后需重启对应 CLI 或 IDE，使新技能被加载。

2.4 在 OpenCode 中接入（详细步骤）

OpenCode 属于脚本接入的平台。以下步骤在 Windows 与 macOS/Linux 两种环境下分别给出，产物与验证方式一致。

2.4.1 前置条件

- OpenCode 已安装并可正常启动；
- 系统已安装 Git，且能访问 GitHub；
- Windows 下使用 PowerShell 执行安装命令；macOS 与 Linux 下使用 bash 与 curl；
- MySQL 等额外依赖不需要；脚本本身不需要管理员权限执行（Windows 上脚本创建的是 NTFS junction，即指向目标的目录链接点，普通用户权限即可创建；macOS/Linux 创建符号链接同样无需特权）。

2.4.2 Windows 安装步骤（PowerShell）

方式一，交互式：执行 2.3 节的 Windows 命令，脚本提示平台编号列表，输入 `opencode` 对应的编号并回车。

方式二，传参：先下载脚本再运行，便于看到脚本文件本身，也便于复用。

```
iwr -useb https://raw.githubusercontent.com/Egonex-AI/Understand-Anything/main/install.ps1 -OutFile install.ps1
.\install.ps1 opencode
```

若系统执行策略阻止脚本运行，可在单次以旁路策略启动的 PowerShell 进程内执行，避免更改全局策略，用毕即恢复默认约束：

```
powershell -ExecutionPolicy Bypass -File .\install.ps1 opencode
```

2.4.3 macOS 与 Linux 安装步骤

方式一，交互式：执行 2.3 节的相应命令，按提示输入 `opencode` 对应的编号。

方式二，传参：

```
curl -fsSL https://raw.githubusercontent.com/Egonex-AI/Understand-Anything/main/install.sh | bash -s opencode
```

2.4.4 安装产物解读

安装结束后，本机出现三类产物。以下路径中 Windows 以 `%USERPROFILE%` 代指用户主目录，macOS/Linux 以 `$HOME` 代指。

产物路径	作用
`~/.understand-anything/repo`	仓库完整克隆，是技能内容的唯一实体；更新时在此执行拉取
`~/.agents/skills/<技能名>`	每个技能目录一个链接（macOS/Linux 为符号链接，Windows 为 junction），把技能暴露给 OpenCode
`~/.understand-anything-plugin`	指向插件根目录的通用链接，其他工具或流程可统一引用

被链接的技能共 9 个，名称为：`understand`、`understand-chat`、`understand-dashboard`、`understand-diff`、`understand-domain`、`understand-explain`、`understand-figma`、`understand-knowledge`、`understand-onboard`。其中 `understand-figma` 未包含在官方手册中，

其用法以仓库内该技能自述为准。

两点补充说明。其一，`gemini`、`codex`、`opencode`、`pi` 四个平台在官方脚本中共用同一个技能目标目录 `~/.agents/skills`，因此一次安装可同时服务这几个平台，重复执行安装脚本亦无副作用。其二，仓库根部的 `package.json` 将主入口字段声明为 `.opencode/plugins/understand-anything.js`；按核实日期检索仓库目录树未发现该路径，该声明指向的文件形态以仓库后续更新为准。官方 README 对 OpenCode 的推荐接入方式为上述脚本链接方式；技能在 OpenCode 会话内的使用方式见 2.5 节。

2.4.5 验证安装

Windows：

```
Get-ChildItem "$HOME\.agents\skills"
```

预期输出：目录列表中出现上述 9 个 `understand*` 名称（以实际运行为准）。macOS/Linux：

```
ls ~/.agents/skills
```

验证链接实体是否存在可用以下命令；输出应指向 `~/.understand-anything/repo`（Windows 显示为 junction 目标，macOS/Linux 显示为符号链接路径）：

```
Get-Item "$HOME\.agents\skills\understand" | Select-Object LinkTarget
```

最后重启 OpenCode，使新技能被加载（此为安装脚本的官方指引）；重启后若技能仍不可见，按 2.5.6 节排查。

2.4.6 调用方式概览

在 OpenCode 中，Understand Anything 以技能（skill）形态注册，会话内的触发不依赖斜杠命令，而以技能提及或自然语言请求实现；技能进入 agent 会话的机制、参数表达、权限控制与排查方法见 2.5 节。安装并重启后，可先做一次最小实验验证安装通道：在提示文本中输入 `@understand`，若技能正文被载入且无加载失败提示，说明安装与发现链路正常，后续可按第 3 章开始首次分析。

2.4.7 更新与卸载

更新通过重新拉取仓库实现，链接本身无需重建：

```
./install.sh --update
```

```
& "$HOME\.understand-anything\repo\install.ps1" -Update
```

卸载移除技能链接与通用链接，但保留仓库克隆（其他平台可能仍在使用）：

```
./install.sh --uninstall opencode
```

```
& "$HOME\.understand-anything\repo\install.ps1" -Uninstall opencode
```

确认不再需要仓库副本后，可手动删除：

```
rm -rf "~/.understand-anything"
```

```
Remove-Item -Recurse -Force "$HOME\.understand-anything"
```

2.4.8 OpenCode 场景常见问题

现象	检查方向	处置
以斜杠输入 `/understand` 无技能响应	OpenCode 不以斜杠触发技能	改按 2.5.2 节以 `@understand` 提及或自然语言请求调用
技能目录为空	安装是否实际完成	重跑安装脚本；脚本对已存在的链接为幂等操作
Windows 下删除产物被拒	目标可能是真实目录而非 junction	脚本自身会拒绝删除真实目录并给出警告；确认后手动处理
需要自定义仓库位置	环境变量 `UA_DIR`	覆盖克隆目标路径，详见 2.6 节

2.5 在 OpenCode agent 会话中的使用

OpenCode 中，Understand Anything 不以独立进程或命令行工具形态运行，而是作为技能进入 agent 会话：会话可用的技能由发现机制注册，模型在需要时以自身 `skill` 工具加载技能正文并按其中指令工作。本节依据 OpenCode V2 官方技能文档与本章安装脚本核实（核实日期 2026 年 10 月 23 日），给出从可见性、触发、参数表达到权限与排查的完整链条。

2.5.1 技能如何进入会话

OpenCode 自动扫描多组技能源。对 `opencode` 平台，安装脚本把技能链接装入 `~/.agents/skills`，该目录是 OpenCode 声明的全局兼容技能源，因此安装后无需在 OpenCode 配置中再登记路径。OpenCode 的技能源优先级由低到高如下，同一 ID 由后注册者覆盖先注册者：

优先级（低到高）	来源
1	OpenCode 内置技能
2	`.claude/skills`（全局来源，再从工程根向当前目录逐级）
3	`.agents/skills`（同上；Understand Anything 的安装目标在此层）
4	`~/.config/opencode/skills`
5	项目 `.opencode/skills`
6	配置文件 `skills` 数组登记项

技能注册后，OpenCode 在每个模型执行步为可用技能向模型提供三要素：技能 ID、显示名称与描述；正文不随列表下发。这意味着 agent 在会话中先看见技能的存在，再按需求决定是否加载，主动触发与否由模型判断，不构成保证。

2.5.2 会话内的三种触发方式

方式一，`@` 显式提及（推荐用于意图明确的调用）。在提示文本中提及技能 ID，OpenCode 立即把该技能正文载入会话，同一条提示中的其余文本即为用户请求：

```
@understand
```

```
@understand-chat 支付流程如何运转?
```

```
@understand-explain src/auth/login.ts
```

技能 ID 由文件路径派生、与目录名一致且区分大小写，不取决于技能封面里的显示名称。本插件在该技能源下注册的 9 个 ID 为：`understand`、`understand-chat`、`understand-dashboard`、`understand-diff`、`understand-domain`、`understand-explain`、`understand-figma`、`understand-knowledge`、`understand-onboard`。

方式二，自然语言请求。以自然语言点名技能，模型按 ID 经 `skill` 工具加载后执行。官方 README 的原文示例为 Use the understand skill to analyze this project；等效中文表达如下：

```
使用 understand 技能分析本项目
```

方式三，模型自主触发。模型根据任务与技能描述的匹配度自行加载，无需使用者点名。适用前提是任务描述与技能描述相关联；因其属模型判断，明确意图时建议采用方式一或方式二。

三种方式的共同前提有二：其一，会话所使用的 agent 具备相应的技能加载权限（见 2.5.5 节）；其二，技能带有 description 字段且未被可见性开关隐藏，被隐藏的技能不出现在模型可用列表中，但仍可经 `@` 提及按 ID 显式加载（见 2.5.5 节的可见性控制）。

2.5.3 会话中表达要求与参数

本插件各项参数（如 `--language`、`--auto-update`、子目录范围与 `--review`）在斜杠平台示例中以命令行样式书写。在 OpenCode 会话中，同一要求以提示文本表达即可，技能正文会按官方指令约定解读。常见映射如下：

斜杠平台示例	OpenCode 会话内等价表达
`/understand --language zh`	分析本项目生成知识图谱，所有产出使用简体中文
`/understand src/frontend`	使用 understand 技能，分析范围限定为 `src/frontend` 子目录
`/understand --auto-update`	为本项目启用 understand 技能的提交后自动更新
`/understand --review`	使用 understand 技能，本次分析开启全量 LLM 复核

两条例行说明。其一，首次运行后语言选择会写入项目内 `.ua/config.json`（见第 3 章），此后各会话不再需要重复说明语言要求。其二，若自然语言表达未被技能按预期解读，可在提示中补写官方参数的原样文本（如 `--language zh`），两种表达方式可并用；具体效果以实际运行为准。

2.5.4 技能加载后发生什么

按 OpenCode 官方文档，`skill` 工具加载按固定次序执行：确定以该 ID 注册的当前版本；校验当前 agent 的技能加载权限；将技能正文（剔除封面元数据）加入会话；同时提供技能所在目录及至多 10 个支撑文件路径的样本。支撑文件不会自动读入，其内容按技能正文中的路径指引由 agent 按需读取；文内路径相对技能目录解析。

对使用者而言，可观察的结果是：加载后会话中出现技能操作指令全文，agent 随后以自身具备的工具（读写、检索、执行等）执行第 3 至第 8 章所述的分析与查询任务；多智能体并发、批次等内部特性（第 9 章）由技能与流水线决定，OpenCode 侧不再参与。技能执行期间产生的交互问题（如首次运行的语言确认，见 3.4 节）同样在会话内以对话形式呈现。

2.5.5 权限与可见性控制

OpenCode 以 `skill` 动作按技能 ID 控制加载行为，规则自上而下执行、最后一条匹配优先，效果分三档：`allow` 不经确认即 advertise 并加载；`ask` 允许 advertise 并加载但在加载前请求确认；`deny` 将技能从模型的可用列表隐藏并拒绝加载。以下示例允许 Understand Anything 全部技能自由加载，其余技能一律询问：

```
{
  "$schema": "https://opencode.ai/config.json",
  "permissions": [
    { "action": "skill", "resource": "understand*", "effect": "allow" },
    { "action": "skill", "resource": "*", "effect": "ask" }
  ]
}
```

该规则组也可置于 `agents.<id>.permissions` 之下，仅对某个 agent 生效，以实现角色差异，例如评审 agent 只保留影响面命令：

```
{
  "$schema": "https://opencode.ai/config.json",
  "agents": {
    "reviewer": {
      "permissions": [
        { "action": "skill", "resource": "understand-diff", "effect": "allow" },
        { "action": "skill", "resource": "understand*", "effect": "deny" }
      ]
    }
  }
}
```

上述两例为展示机制之用，`reviewer` 非官方预设 agent 名称，具体 agent 配置以 OpenCode 官方文档为准。可见性开关方面：技能封面元数据设 `metadata.opencode/autoinvoke: false` 或 `disable-model-invocation: true` 时，该技能从模型可用列表中隐藏，但对 `@` 提及的显式加载不影响。Understand Anything 技能封面的开关状态未在核实范围内，遇到按 ID 提及仍以自然语言无法触发时应以技能文件与权限条目为排查入口。

2.5.6 技能 ID 冲突与不可用排查

按 OpenCode 官方排查顺序，技能缺失或加载内容不符时依次检查六项：文件形态（技能源根下的 `*.md`，或分组目录内名为 `SKILL.md` 的文件）；ID 是否为路径派生值且区分大小写；技能是否具备 description；`opencode/autoinvoke` 与 `disable-model-invocation` 的设置；是否存在同 ID 的更高优先级来源；当前 agent 的技能权限条目。

对 Understand Anything 而言，最常见的两类冲突来源：一是使用者向 `~/.config/opencode/skills` 或项目 `.opencode/skills` 手工复制过同 ID 的旧版技能；二是项目内有实测同名技能（如自行编写的 `understand.md`）。确认覆盖后删除多余来源即可恢复安装脚本版本的行为。OpenCode 侧的通用故障定位（服务状态、日志路径、最小复现）可按 OpenCode 官方排查指引执行，本教程不展开。

2.6 其他平台接入要点与高级选项

以下为官方 README 所载各平台的补充接入命令。

平台	命令或操作	调用差异
Cursor	克隆本仓库后打开，自动发现 `.cursor-plugin/plugin.json`；或 Cursor Settings → Plugins 中粘贴仓库地址手动添加	多数技能以斜杠调用
VS Code + Copilot	克隆本仓库后打开，自动发现 `.copilot-plugin/plugin.json`（v1.108+）；个人级技能可用脚本传参 `vscode`	多数技能以斜杠调用
Copilot CLI	`copilot plugin install Egonex-AI/Understand-Anything:understand-anything-plugin`	无特殊
Kiro CLI	脚本传参 `kiro`；随后 `kiro-cli chat --agent understand "Analyze this project"`	无斜杠，走专属 agent
Kiro IDE	脚本传参 `kiro`；技能链接进 `~/.kiro/skills/`，agent 写入 `~/.kiro/agents/understand.json`	重启 IDE 生效
Codex	脚本传参 `codex`	前缀为 `$`，即 `$understand`

脚本高级选项：环境变量 `UA_REPO_URL` 覆盖克隆来源（内网镜像场景），`UA_DIR` 覆盖克隆目标（默认 `~/.understand-anything/repo`）。

2.7 安装验证自检清单

无论何种平台，安装完成的共同判据为：仓库克隆目录存在且含 `understand-anything-plugin`；目标技能目录中出现技能链接或技能目录；重启助手后技能可被实际调用。OpenCode 场景的调用验证采用 `@understand` 提及或自然语言请求；命令式平台验证斜杠命令可用。任一环节缺失，回到 2.4.8 节与 2.5.6 节对照排查。

本章要点：接入方式三路并存；`~/.understand-anything/repo` 是唯一实体，链接是通道；OpenCode 读 `~/.agents/skills` 属全局兼容技能源；OpenCode 会话内调用不使用斜杠，以 `@` 提及、自然语言请求或模型自主触发实现；`gemini`、`codex`、`opencode`、`pi` 共享技能目录；Codex 前缀为 `$`。

第 3 章 首次分析：生成知识图谱

安装完成后，第一项任务是让流水线扫描整个项目并产出图谱。这一步是全部后续能力的基础，也是 token 开销最大的一步；本章说明如何评估成本、如何执行、如何指定语言，以及如何确认落盘结果。

3.1 成本评估与本地模型选择

官方文档明确提示：首次 `/understand` 分析整个代码库，在大型项目上可能消耗可观的 token 数量。降低成本的两条路径如下。

- 使用订阅型或按量计费的模型计划，接受首次全量分析的成本；
- 将平台指向本地模型提供方（如 Ollama）。官方将此列为隐私或企业场景的推荐方案，并提示按 Ollama 侧的集成文档修改平台模型提供方；本地模型用于初始化分析可绕开云端 token 计费。

后续重复运行的增量机制（见第 6 章）使分析成本仅与变更文件数相关，首次成本是一次性支出。

3.2 执行 `/understand`

在项目根目录（即代码库顶层）下执行：

```
/understand
```

流水线的工作顺序为：扫描项目、识别语言与框架；抽取每个文件、函数、类与依赖；构建知识图谱并保存至 `.ua/knowledge-graph.json`。并行执行细节与各智能体职责见第 9 章。

3.3 输出与数据目录

图谱数据默认落在项目内 `.ua/` 目录，主编谱文件为 `.ua/knowledge-graph.json`。一个兼容性事实：已存在 `.understand-anything/` 目录的项目将继续以该目录为数据目录，无需迁移；两套目录名对应新旧版本，正文后续内容以新命名 `.ua/` 表述。

3.4 语言本地化：`--language` 与 `.ua/config.json`

`/understand` 支持以 `--language` 参数指定产出语言，影响三类内容：图谱节点的摘要与描述、Dashboard 的界面标签与提示、导览讲解文字。官方列出的支持值为：

代码	语言
`en`	英文（默认）
`zh`	简体中文
`zh-TW`	繁体中文
`ja` 、 `ko` 、 `ru` 、 `vi`	日语、韩语、俄语、越南语

以中文产出为例：

```
/understand --language zh
```

首次在项目内运行时，若未传 `--language` 且尚无已存语言配置，`/understand` 会检测当前会话语言：非英文会话会先请求确认或改写，英文会话不受影响。选择结果写入项目内 `.ua/config.json`，之后每次运行自动沿用，无需重复指定。

3.5 增量机制的预告

首次运行之后，重复执行 `/understand` 默认只重新分析自上次运行以来有变更的文件。机制细节与自动化选项在第 6 章展开。

3.6 运行结果确认

官方文档未固定展示控制台完整输出，建议以两项事实判据确认成功：其一，`.ua/knowledge-graph.json` 已生成且非空；其二，执行 `/understand-dashboard`（见第 4 章）能在浏览器中打开图谱。若分析中途中断，重新执行 `/understand` 即可，已完成部分可经增量机制复用。

本章要点：首跑全量、重跑增量；语言配置一次写入 `.ua/config.json` 后长期沿用；成功判据看文件落盘与 Dashboard 可开。

本章要点之外还应记录一个边界：官方脚本与文档未说明分析耗时上限的硬性保证，大型仓库首跑可能出现较长等待，属预期行为而非常见故障。

第二篇 掌握篇

第 4 章 交互式 Dashboard

分析完成后的核心交互界面是 Dashboard。本章按浏览、检索、导览、视角四组功能说明其使用方式；所有功能均以官方 README 描述为准，具体控件布局以在线演示为准。

4.1 启动

```
/understand-dashboard
```

执行后打开交互式网页面板，将代码库渲染为图谱：按架构分层着色、支持搜索与点击浏览。点击任一节点可查看其源码、关系与自然语言解释。

想在不安装任何工具的情况下了解面板形态，可直接访问官方在线演示 `https://understand-anything.com/demo/`，浏览器内即可平移、缩放、搜索与探索。

4.2 结构图浏览与节点详情

结构图中文件、函数、类均为可点击节点。选中节点后，面板提供三类信息：节点源码、与其他节点的关系连线、该节点用途的日常语言摘要。摘要由 LLM 语义层生成，描述的是该文件为何存在，而不仅是它引入了什么。

4.3 架构分层与着色

面板将节点自动归入五类架构分层，并用颜色图例区分：

分层	典型内容
API	对外接口与路由入口
Service	业务处理逻辑
Data	数据访问与存储
UI	展示层与交互界面
Utility	通用工具与辅助函数

分层归属由 `architecture-analyzer` 智能体在分析阶段判定（见第 9 章）。分层视角的价值在于快速识别边界污染：例如 Data 层节点直接被 UI 层引用，通常提示分层约束缺失。

4.4 模糊与语义搜索

搜索支持两种模式：按名称匹配（模糊查找标识符），以及按语义查找。语义搜索官方给出的问题示例为询问"哪些部分处理认证（auth）"，返回结果是图谱中相关的节点集合，而非仅文件名匹配。语义能力来自 LLM 写入每个节点的摘要与标签。

4.5 导览

导览由 `tour-builder` 智能体生成：以依赖顺序排列的架构讲解清单，规定按合适的先后顺序阅读代码库。对目标为"先骨骼后血肉"的读者，建议先完成导览再自由浏览。

4.6 语言概念与角色视角

语言概念：分析会在代码出现处标注 12 种语言编程模式（如泛型、闭包、装饰器）并随文解释。该功能面向不熟悉某语言特性的读者，在同一图谱中按需展开。

角色自适应界面：面板依据使用者身份调节信息详略，官方列出的角色档位为初级开发、项目经理（PM）与重度用户（power user）。切换视角的标准建议：确有影响面板行为的版本或配置项官方未另行说明，具体切换方式以在线演示与面板内实际控件为准。

4.7 无 AI 环境查看的预告

若图谱已生成并提交，团队成员运行一次命令即可在浏览器打开同一面板，无需 AI 助手与模型调用，该通道在 7.4 节展开。

本章要点：Dashboard 的四组能力——浏览（节点详情、分层着色）、检索（名称与语义）、导览（依赖顺序讲解）、视角（语言概念、角色档位）；入口统一为 `/understand-dashboard`。

第 5 章 日常核心命令

图谱生成后的日常使用集中在三条命令上：询问、解释与变更影响。三者覆盖"读代码、懂代码、改代码"三个高频时刻；更进阶的命令在后续章节说明。

5.1 `/understand-chat`：向图谱提问

该命令以图谱为语境回答问题，答案能定位到具体文件、函数与调用关系。

```
/understand-chat How does the payment flow work?
```

预期行为：官方示例问题即"支付流程如何运转"；回答内容为依据图谱组织的讲解，具体文本以实际运行为准。适用场景：理解某流程全局走向、确认某功能由哪些文件支撑。

5.2 `/understand-explain`：深入单个文件或函数

```
/understand-explain src/auth/login.ts
```

参数为项目内相对路径。产出为该路径的深度讲解，粒度深于节点的默认摘要。适用场景：准备修改某个文件之前，先掌握其职责与关联。

5.3 `/understand-diff`：当前变更的影响面

```
/understand-diff
```

该命令基于当前未提交的变更，展示这些改动对系统其余部分的影响关系。官方对其定位的表述为：提交之前看清改动波及何处，理解涟漪效应（ripple effects）。第 11 章将其纳入代码评审工作流。

5.4 命令对照与选用建议

命令	参数形态	典型问题	使用时机
`/understand-chat`	自由问题文本	流程如何运转	全局理解
`/understand-explain`	文件或函数路径	这个文件做什么	修改前定点理解
`/understand-diff`	无	这次改动影响谁	提交或评审前

选用理由：`chat` 面向模糊问题，`explain` 面向已锁定路径，`diff` 面向已有变更；问题越模糊越宜用 `chat`，问题越具体越宜用 `explain`。

本章要点：日常命令三条、参数形态各一；建议形成"diff 先行"的习惯——改动前先看影响面再动手。

第 6 章 图谱维护与自动化

代码库持续演进，图谱随之过时。本章说明图谱的时效性如何维护：增量机制控制单次开销，提交钩子保持图谱与代码同步，范围限定控制大型仓库的分析边界。

6.1 增量更新机制

重复执行 `/understand` 默认为增量模式：仅重新分析自上次运行以来发生变化的文件。变更识别基于静态解析引擎生成的指纹（对文件结构内容的可复现标识，同一输入恒得同一指纹，详见第 9 章）。因此日常维护成本仅与实际变更量成正比。

6.2 `--auto-update` 提交钩子

```
/understand --auto-update
```

该参数为项目配置一个 post-commit 钩子（Git 在每次 commit 完成后自动执行的脚本）：每次提交后增量修补图谱，使每笔提交落地时图谱保持同步。选择建议：希望以自动化换取一致性时启用；若团队节奏允许，也可只在发布前手动重跑 `/understand`。两者的取舍点在于图谱 staleness（过时程度）的可接受区间，属团队约定而非技术限制。

6.3 子目录范围限定

```
/understand src/frontend
```

参数为仓库内路径，分析范围收窄至该子目录。官方将其列为超大 monorepo（多仓合一的代码仓库形态）的处理手段：按需要理解的目录分区分析，避免整仓成本。

6.4 维护节奏建议

依据已核实的机制，本教程的维护建议为三条，并每条给出理由：其一，活跃开发期启用 `--auto-update`，理由是每次提交自动同步，图谱不会在关键节点过时；其二，发布前手动重跑一次全量 `/understand`，理由是发布节点的图谱质量值得一次成本支出；其三，monorepo 团队按目录限定范围，理由是分区成本远低于整仓首跑，且多数读者只关注所辖目录。

本章要点：增量机制的运行基础是指纹比对；自动化由 `--auto-update` 承担；范围限定 `src/frontend` 式传参解决大仓问题。

第三篇 高级篇

第 7 章 团队协作与图谱共享

图谱的本质是 JSON 文件，这使它具备与代码同等的可版本化能力：提交一次，全队受益。本章给出官方的提交规范、大文件处理方案，以及不需要 AI 助手的浏览通道；这是全员可用的关键。

7.1 共享模型的原理

流水线产出的图谱是确定性与语义数据的组合（确定性部分同一输入恒得同一输出）。提交图谱后，队友浏览只需要 Dashboard 通道，不再需要模型调用与分析时间。官方将其价值归纳为三项：入职引导、PR 评审与 docs-as-code（以文档形态随代码演进管理的实践）。

官方给出一个可参照的真实示例：`GoogleCloudPlatform/microservices-demo` 仓库提交了其图谱，覆盖 Go、Java、Python、Node 多语言参考实现。

7.2 提交内容与排除项

应提交 `.ua/` 目录下的全部内容，但官方明确列出两个本地暂存性质的例外：

```
.ua/intermediate/
.ua/diff-overlay.json
```

其中 `intermediate/` 属流水线中间产物，`diff-overlay.json` 属本地影响面分析的过程数据；两者由各成员本地生成即可。旧版项目若数据目录为 `.understand-anything/`，则将上述规则中的目录名做相应替换。

7.3 大图谱与 git-lfs

10 MB 以上量级的图谱建议用 Git LFS（Large File Storage，Git 大文件存储扩展）追踪，避免普通文本模式下的大文件入库负担：

```
git lfs install
git lfs track ".ua/*.json"
git add .gitattributes .ua/
```

7.4 免 AI 环境的浏览通道：npx viewer

图谱提交之后，任何成员只需 Node.js 18 或更高版本即可打开完整面板，无需 Claude Code、无需模型、无需 API Key：

```
npx https://github.com/Egonex-AI/Understand-Anything/releases/latest/download/understand-anything-viewer.tgz
/path/to/analyzed/project
```

运行行为以官方文档描述为准：终端输出一个带 token 的本地地址（形如 `http://127.0.0.1:5173/?token=…`）并自动打开浏览器。目标目录默认取当前目录，目录内必须存在已提交的数据目录（`.ua/`，或旧版 `.understand-anything/`）。官方对安全性的表述为：全部内容从本地磁盘只读提供，无 LLM 调用，数据不出本机。

7.5 从源码构建的浏览通道

习惯从源码开发或需要调试 Dashboard 的读者，可在本仓库克隆中采用构建通道：

```
pnpm install
pnpm --filter @-understand-anything/core build
GRAPH_DIR=/path/to/analyzed/project pnpm dev:dashboard
```

其中 `GRAPH_DIR` 为环境变量，指定待浏览项目所在的图谱数据目录；该通道经 Vite 开发服务器提供同一面板。

7.6 团队工作流建议

就官方已核实的机制而言，本教程的建议为：将 `.ua/` 纳入提交规范（排除项照 7.2 节），启用 6.2 节钩子保持图谱与代码同步，评审与入职场景使用 7.4 节通道全员开放浏览，形成一条从分析到共享的最短闭环。理由是这一组合不增加任何云端调用成本，且各环节均以本地数据服务。

本章要点：图谱就是 JSON 的可提交产物；排除 `intermediate/` 与 `diff-overlay.json`；大图谱走 git-lfs；全队浏览只依赖 Node.js 18 与一条 npx 命令。

第 8 章 业务与知识库分析

除代码结构外，Understand Anything 还向两个方向延伸：面向产品的业务逻辑提取，与面向文档知识库的语义建网。两者各有一个专属命令与专属智能体支撑，本章分别说明。

8.1 `/understand-domain`：提取业务域

```
/understand-domain
```

该命令在结构图谱之上补充业务层信息：划分业务域（domain），梳理业务流程（flow）与流程步骤（step）。产出在 Dashboard 的域视图（domain view）中以水平布局展示代码与实际业务过程的对应关系；视图中域、流程与步骤依次排布。支撑智能体为 `domain-analyzer`，属流水线中第 6 个智能体，仅在该命令调度时参与（见 9.2 节）。

适用场景为：确认代码结构所映射的业务能力、核对流程是否与产品说明一致、梳理跨域调用点。与 `chat` 的区别在于：`chat` 回答具体问题，`domain` 产出成体系的业务层图谱。

8.2 `/understand-knowledge`：分析 wiki 知识库

```
/understand-knowledge ~/path/to/wiki
```

参数为知识库根目录路径。官方针对的对象为 Karpathy 式 LLM wiki——即社区公开讨论的以 `index.md` 为入口、以 wikilink（条目互链格式）组织的一类 Markdown 知识库，参考格式见官方 README 所附 Karpathy 的 gist 链接。

工作流程分两段：确定性解析器从 `index.md` 等文件中抽取 wikilink 与分类；随后由第 7 个智能体 `article-analyzer` 从文章中抽取实体（entity，指被讨论的对象与概念）、主张（claim，指文中提出的可核观点）与隐式关系，使隐含内容浮出。产出为力导向布局的知识图谱，并做社区聚类（community clustering，指联系紧密的节点聚团呈现），点此库成为可导航的互链网络。

8.3 执行要点与边界

两项注意。其一，两条命令均依赖已安装的技能环境（第 2 章）。在任意支持平台上的命令形态一致；OpenCode 等脚本接入平台的会话内调用方式见 2.5 节。其二，`/understand-domain` 的水平视图与 `/understand-knowledge` 的力导向视图在同一个 Dashboard 中的具体切换入口，官方 README 未固定描述，以在线演示与实际面板为准。

本章要点：业务分析与知识库分析各有专属命令；业务视图对应代码域，知识视图对应文档实体；两类产出扩展了图谱的语义维度。

第 9 章 原理与复核机制

掌握使用的读者需要向下追问：图谱为何可信、开头为何不慢、结果为何可复现。本章拆解静态解析与 LLM 的分工、多智能体流水线的编排，以及产出质量的复核机制。

9.1 混合引擎：Tree-sitter 与 LLM 的分工

解析层采用 Tree-sitter（一种以 C 语言编写、以语法规则生成具体语法树的通用解析框架）做确定性静态解析，抽取导入、导出、函数与类定义、调用点、继承关系等结构事实；这些事实在扫描阶段预解析为结构映射表 `importMap`，供后续文件分析环节直接引用，避免重复推导。指纹增量也以解析层输出为基础（见 6.1 节）。

语义层由 LLM 承担，完成解析器无能为力的四类工作：节点日常语言摘要与标签、架构分层归属、业务域映射与导览讲解，以及语言概念的随文标注。

官方对这种分工的因果表述为：结构属可复现（同一代码恒得同一边集），语义属意图捕获（文件的存在目的，由此解释其行为）。因此结论是可信度分层：结构数据可信度最高，语义摘要应以批判阅读对待。

顺带核实解析层的语言覆盖：仓库构建配置中编入的 Tree-sitter 语法包覆盖 C、C++、C#、Go、Java、JavaScript、TypeScript（含 TSX）、PHP、Python、Ruby、Scala 共十一族语言。官方 README 未以清单形式强调此点，本条依据为构建配置文件，实际支持范围以运行时为准。

9.2 多智能体流水线编排

`/understand` 编排 5 个智能体，`/understand-domain` 增至 6 个，`/understand-knowledge` 增至 7 个，职责分工如下。

智能体	职责	归属命令
`project-scanner`	发现文件，识别语言与框架	`/understand`
`file-analyzer`	抽取函数、类与导入；产出节点与边	`/understand`
`architecture-analyzer`	判定架构分层	`/understand`
`tour-builder`	生成导览讲解	`/understand`
`graph-reviewer`	校验图谱完整链与引用一致性（默认随流程执行；全量 LLM 复核需 `--review`）	`/understand`
`domain-analyzer`	抽取业务域、流程与步骤	`/understand-domain`
`article-analyzer`	从 wiki 文章中抽取实体、主张与隐式关系	`/understand-knowledge`

并行效能为分析速度的主要来源：文件分析智能体间并行，官方给定的上限为 5 个并发工作者、每批 20 至 30 个文件。

9.3 复核机制：默认随流程与 `--review`

`graph-reviewer` 默认随流水线执行，属轻量校验，关注图谱的结构完整性；`--review` 开启全量 LLM 复核，属质量上限提升。本条选择建议并理由：一般情况下用默认档即可；对将纳入团队共享或文档化的图谱，宜在生成阶段或发布前以 `--review` 重跑首回分析，理由是共享图谱的潜在错误面会被全员放大，一次性质量投入的收益与使用面成正比。

9.4 技能清单一览

技能目录截至核实日共 9 项，官方手册明确叙述 8 项；`understand-figma` 见 2.4.4 节说明。各技能对应的命令调用名即第 3 至第 8 章所述各命令，此处不重复列表。

本章要点：结构可复现、语义属意图，两者由不同引擎产出；5 个并发文件分析智能体是速度来源；复核有默认与全量两档，共享图谱适用全量。

第四篇 精通篇

第 10 章 大型仓库实战与成本优化

大型仓库（monorepo、多语言混合仓库、百万行级别项目）最能检验对成本与策略的把握。本章给出收敛到四条的实战清单，每一条的依据均来自已核实机制；具体数值上限官方未给出，不做硬性承诺。

10.1 monorepo 分区分析

超大仓库按注意力范围分区分析：先 `/understand src/frontend`，需要时再补其他目录。收益来自首跑成本仅与所选范围成比例，团队各部可用各自关注的分区图谱先行工作。注意点：分区对应同一工具的子范围，而非多个独立数据目录；分区分析的目录间关系如何转归主图谱，官方文档未另行说明，建议以当次运行结果为准。

10.2 成本结构四因子

依据官方文档，token 开销的控制因子为：首跑全量（成本中心）、订阅或本地模型（支付方式）、共享图谱跳过流水线（对团队的重复成本归零）、增量机制（单次维护成本回落）。四因子联合作用的最优路径按团队情况排序：多成员团队优先共享图谱，因流水线成本一次发生；单机用户优先增量与自动化钩子，因首跑之后边际成本有限。

10.3 并行与批次时间的理解

首跑耗时由 9.2 节参数决定上限：5 个并发工作者与每批 20 至 30 个文件，则单批等待随批次序列排布；瓶颈不再是逐文件串行模型调用，而是批次间的等待。由此可推出经验倾向，本处仅列为推断：固定模型与仓库规模下，首跑耗时随文件数近似线性增长，与图规模增长不同——属机制推论，以实际运行为准。

10.4 实战检查清单

检查项	判断内容
仓库是否已分区	是否能确定大多数读者关注的目录范围
首跑预算是否明确	是否已决定云端计划或本地模型初始化
数据目录规范是否建立	`.ua/` 是否已纳入提交与排除项规则
浏览通道是否全员可用	Node.js 18 环境是否覆盖团队
自动化是否选档	全量首跑后选择 `--auto-update` 还是发布前手动重跑
复核是否到位	共享图谱是否经 `--review` 生成

本章要点：分区控制范围、共享摊薄成本、增量控制边际、复核提升质量上限。四者配合的结果是大型仓库上工具不再昂贵，而成为常规产能。

第 11 章 融入开发工作流

工具的价值由工作流承载。本章给出以已核实命令组合的三个典型工作流：新人入职引导、代码评审与文档协作，并说明与 AI 编码助手日常工作的搭配方式。

11.1 新人入职引导：`/understand-onboard`

```
/understand-onboard
```

该命令生成面向新成员的上手资料。与图谱静态呈现的关系：配合导览形成完整引导，先入职资料后导览讲解，新人无需依赖资深同事逐个答疑。建议衔接的一步为：新成员随后以 `chat` 轮询式提问，将不理解处即时消化。

11.2 代码评审：`/understand-diff` 与共享图谱对照

评审场景的标准组合：提交者以 `diff` 看影响面再提交；评审者依共享图谱（7.4 节通道）走入受影响节点，以 `chat` 提复述问题。此流程的理由在两个落差被抹平：提交者的涟漪预估落差与评审者的上下文切换落差；图谱的语义摘要降低了评审者读新文件的时间成本。

11.3 文档协作：docs-as-code 组合

提交 `.ua/` 的实践使图谱成为文档资产。官方对其定位的表述为 docs-as-code 实践的一部分。两个具体应用：PR 说明引用图谱节点作为背景；设计变更前以 `domain` 视图核对业务流程不再漂移。

11.4 与 AI 编码助手的搭配

图谱对助手侧的价值为：助手获得全局结构知识，减少对既陌生又无界面支持的仓库态试错。搭配建议来自官方既有提示的两处复用：本地模型方案（3.1 节）用于初始化与隐私敏感项目；技能提及与自然语言双通道（2.5 节）覆盖 OpenCode 等技能式平台的调用习惯。

本章要点：`onboard` 治门槛、`diff` 治盲改、docs-as-code 治漂移；三者均以已生成的图谱为底座，不产生额外分析任务。

第 12 章 从使用者到贡献者

Understand Anything 为开放项目，欢迎贡献。本章给出仓库工程面、贡献流程与常见问题排查，使长期使用者具备参与或自行排障的能力。

12.1 仓库工程面概览

仓库内与本教程直接相关的目录与文件如下（以核实日为准）：

路径	内容
`understand-anything-plugin/skills/`	9 个技能目录，各含技能指令文件
`understand-anything-plugin/agents/`	各智能体定义文件
`install.sh` 、 `install.ps1`	双平台安装脚本，2.3 至 2.4 节所述行为的唯一实现
`README*s/`	简体、繁体中文及日、韩、西、土、俄、越多语言 README
`docs/` 、 `homepage/` 、 `tests/` 、 `scripts/`	文档、宣传页与演示、测试与工程脚本
`.claude-plugin/` 、 `.copilot-plugin/` 、 `.cursor-plugin/`	各平台自动发现的清单目录
`package.json`	工程信息；主入口指向 OpenCode 插件文件；无版本号字段

12.2 构建与测试

工程化要求两项：使用 pnpm 包管理器（`package.json` 声明的包管理器版本为 pnpm 10.6.2），语言为 TypeScript（构建配置声明 5.7 系列）。官方给出的核心测试命令为：

```
pnpm --filter @understand-anything/core test
```

其他 `package.json` 已声明的脚本包括：`build`（全仓构建）、`lint`（ESLint 检查）、`dev:dashboard`（本地开发服务器，7.5 节已用）、`benchmark:large-repo`（大仓性能基准脚本）。

12.3 贡献流程

官方贡献指引为四步：fork 仓库；创建功能分支（分支名要求形如 `feature/my-feature`）；跑通核心测试；提交并开启 pull request。对涉及面较大的改动，官方要求先在 issue 中先行讨论再实施。

12.4 社区资源

截至核实日，官方社区入口包括：仓库 issue 与 pull request 区（核实日数字分别为 122 与 210 上下，属动态数据仅作量级参考）、Discussions 区；官方主页与在线演示；多语言 README 群；Better Stack 制作的社区演示视频（YouTube）。制作教程类内容的贡献者可经 issue 或 PR 提请收录，官方声明欢迎此类外部内容。

12.5 常见问题排查

以下表覆盖本教程各章所述机制派生的可核实排查路径；凡官方文档未述及的故障现象不列。

现象	排查方向
安装后技能不可调用	技能链接是否存在（2.4.5 节）；助手是否重启
`@understand` 无法加载技能	同 ID 来源冲突或权限限制
Codex 中斜杠无效	前缀应为 `$`（2.6 节）
首跑耗时显著	大仓属预期；考虑分区（6.3 节）与共享图谱（7.1 节）
图谱过时	是否启用钩子或发布前重跑（6.2 节）
团队浏览失败	Node.js 版本是否 18 或更高；目录内是否有 `.ua/` 数据目录（7.4 节）
提交中大文件	是否配置 git-lfs（7.3 节）
语言产出非预期	`.ua/config.json` 是否已存旧配置；显式传 `--language` 覆盖（3.4 节）
Windows 密钥或策略类错误	脚本执行策略旁路（2.4.2 节）；项确认 junction 与真实目录的区别（2.4.8 节）

本章要点：工程面理解以官方脚本与清单为唯一依据；贡献前先过核心测试；排障走机制而非猜命令。

结语

从零基础视角回顾全部路径：安装与接入（第 2 章）建立技能环境；首次分析（第 3 章）以一次成本建立图谱资产；Dashboard 与日常命令（第 4、5 章）完成每日理解任务；维护与共享（第 6、7 章）把个人工具转为团队资产；业务与知识延伸（第 8 章）拓宽图谱语义；原理（第 9 章）确立对产出的信任边界；大型仓库与工作流（第 10、11 章）把工具嵌进工程实践；贡献（第 12 章）完成从使用者到共建者的转向。指南偏保守之处均因部分内容（如分析时长上限、面板切换布局的临时状态）未见于官方文档；此类内容以官方主页、在线演示与仓库后续更新为准。